

Universidad Tecnológica del Perú
Escuela de Ingeniería
Ing. de Software y Sistemas



Avance de Proyecto que como parte del curso

Herramientas de Desarrollo

"Sistema web para inventario de componentes de
computadoras"

Docente:

Dr. Charles Dummar Camasca Macedo

Presentan los alumnos:

Aulla Gonzales, Jacson Michael

Condori Chauca, Jhosef Nacor

Huauya Aldazabal, Yulissa Leydi

Millan Pariona, Mauricio Leandro

Pomayay Sobero, Joaquín Hernan

Lima - Perú

2025

DEDICATORIA

Le dedicamos este trabajo a nuestros familiares, amigos y compañeros, quienes nos han ayudado a alcanzar nuestros objetivos, apoyándonos y aconsejándonos en circunstancias poco favorables.

AGRADECIMIENTOS

Expresamos nuestro sincero agradecimiento a nuestra docente guía, **Charles Dummar Camasca Macedo**, por su constante apoyo, orientación y dedicación durante el desarrollo de este proyecto.

Índice

Introducción	7
Capítulo I	8
Descripción de la problemática	8
Planteamiento del Problema	8
Objetivos	8
General	8
específicos	8
Justificación	9
tecnic	9
Social	10
Financiera	11
Capítulo II	11
Marco teórico	11
Inventario periódico	11
Inventario permanente o automatizado	11
Antecedentes	12
Metodología de Desarrollo	12
Diagrama de base	14
Capítulo III	15
Requerimientos	15
Funcionales	15
No Funcionales	19
Metodología de desarrollo desplegada	21
Diagrama de clases	22

Diagrama de base de datos	24
Diseño de entorno	24
Estructura y Patrones de Diseño Aplicados	28
Dependencia e Inyección	29
Codificación	30
Capa de Modelo	30
Capa de Repositorio	31
Capa de Servicio	32
Capa de Controlador	33
anexos	34

Índice de figuras

1	Resumen de VAN y TIR	12
2	Carta gantt	13
3	base de datos	14
4	Diagrama de clases	22
5	Diagrama de la base de datos	24
6	MovimientoLinea	30
7	MovimientoLineaRepository	31
8	MovimientoLineaService	32
9	MovimientoLineaController	33
10	Carta gantt	34
11	base de datos	35
12	Diagrama de clases	36
13	Diagrama de la base de datos	37

Resumen

La empresa, dedicada a la comercialización de componentes de PC, atraviesa un proceso de expansión que ha incrementado el número de sedes y el volumen de operaciones. Sin embargo, el control de inventario continúa realizándose de forma manual en cuadernos físicos, lo que provoca registros incompletos o duplicados, demoras en la actualización de existencias, ausencia de información en tiempo real y alta exposición a pérdidas por extravío o deterioro del soporte. La falta de trazabilidad por número de serie y por ubicación dificulta la conciliación entre almacenes y puntos de venta, limita la detección de mermas y devoluciones y reduce la confiabilidad de los datos.

Frente a este contexto, se plantea el desarrollo de un sistema web de inventario centralizado, escalable y seguro, que provea visibilidad en tiempo real del stock en múltiples sucursales y garantice datos consistentes. La solución propuesta se fundamenta en una arquitectura de microservicios, con Spring Boot para el backend, Angular para el frontend y SQL Server como motor de base de datos, privilegiando la trazabilidad, la disponibilidad y la facilidad de mantenimiento. El proyecto se gestionará con Scrum, mediante sprints cortos y entregables incrementales (MVP temprano, pruebas unitarias e integración, despliegue progresivo), asegurando alineamiento continuo con las necesidades del negocio y una adopción ordenada por parte de los usuarios.

Introducción

Este proyecto propone implementar un sistema web de control de almacén centralizado y escalable para reemplazar la gestión manual en cuadernos que hoy provoca errores, baja trazabilidad por serie y ubicación, falta de información en tiempo real y riesgos de quiebres o sobreinventario; la solución—basada en microservicios con Spring Boot (backend), Angular (frontend) y SQL Server—incorpora control de entradas, salidas, transferencias y ajustes, trazabilidad por número de serie, niveles mínimos con alertas, reportes operativos y gerenciales, control de accesos por roles, auditoría e integración con compras y ventas, y se gestionará con Scrum para asegurar entregas incrementales; con ello se busca reducir errores, mejorar el cumplimiento de pedidos y optimizar el abastecimiento con decisiones soportadas en datos confiables; en el presente documento se explicará, en el Capítulo I, la problemática, el planteamiento del problema, los objetivos, la justificación técnica y social y el alcance/arquitectura de la solución; y, en el Capítulo II, el marco teórico, los modelos de inventario pertinentes y la metodología de desarrollo (Scrum) con sus elementos clave.

Capítulo I

Descripción de la problemática

Una empresa dedicada a la comercialización de componentes de PC está en plena expansión, con nuevas sedes en operación. Sin embargo, el almacén general aún controla el inventario de forma manual en un cuaderno, un método ineficiente que genera omisiones de datos, limita la disponibilidad oportuna de la información y aumenta el riesgo de pérdidas.

Planteamiento del Problema

Se plantea desarrollar una solución web de inventario centralizada y escalable que permita a la empresa gestionar de manera eficiente el stock de componentes de PC en múltiples sucursales, con visibilidad en tiempo real y datos consistentes. La plataforma debe unificar la información dispersa y acompañar el crecimiento del negocio sin perder precisión ni oportunidad en los registros.

La solución debe asegurar trazabilidad a nivel de producto y número de serie, control de entradas, salidas, transferencias y ajustes, definición de niveles mínimos con alertas de reposición, generación de reportes operativos y gerenciales, control de acceso por roles y registros de auditoría, además de integrarse con los procesos de compras y ventas. Con ello se busca reducir errores, mejorar el cumplimiento de pedidos, optimizar el abastecimiento y respaldar decisiones oportunas basadas en información confiable.

Objetivos

General

Desarrollar un sistema web para el Control de Almacén para una tienda de componentes.

específicos

- Recopilar información del negocio

- Definir las necesidades del negocio y alcance
- Realizar analisis tecnico y de viabilidad
- Realizar Modelado de Entidades y Arquitectura
- Diseño UX/UI y maquetado
- Planificar el Desarrollo del proyecto
- desarrollar los entregables para cliente
- Realizar pruebas unitaria e integracion
- desplegar la Solucion

Justificación

tecnica

1. Sql server(Motor de base de datos):

- CPU: x64, mínimo 1.4 GHz (recomendado: 2.5 GHz+).
- RAM: 4 GB mínimo (recomendado: 16–32 GB).
- Almacenamiento: 6 GB libres (preferible SSD de 100 GB+).
- Sistema operativo: Windows Server 2016+ o Windows 10 1607+.
- Conectividad: Puerto 1433, codificación UTF-8, acceso vía JDBC

2. Spring Boot (Backend Java):

- CPU: 2 núcleos mínimo (recomendado: 4–8 núcleos).
- RAM: 4 GB mínimo (recomendado: 8–16 GB).
- Almacenamiento: 10 GB libres (preferible SSD).
- Java: Versión 17+.

- Sistema operativo: Windows 10+, Linux Ubuntu 20.04+.
- Dependencias: Spring Web, Spring Data JPA, Spring Security (opcional).
- Conexión: JDBC a SQL Server, configuración CORS para Angular.

3. Angular (Frontend TypeScript):

- CPU: 2 núcleos mínimo.
- RAM: 4 GB mínimo.
- Almacenamiento: 5 GB libres.
- Node.js: v18+.
- Sistema operativo: Windows, macOS o Linux.
- Producción: Archivos estáticos servidos por Nginx, Apache o CDN.

La Arquitectura para el Sistema de Control de Almacén se fundamenta en estas 3 tecnologías clave, estas responden a criterios de escalabilidad, trazabilidad, seguridad y facilidad de mantenimiento, alineados con los objetivos del negocio, por lo que el cliente ha aprobado estos requerimientos.

Social

La implementación del sistema optimiza el trabajo interno al agilizar la búsqueda de productos y reducir errores, estandarizando procedimientos y disminuyendo el estrés operativo. Para las personas clientes, garantiza la disponibilidad por sucursal, acelera la atención y reduce las ventas fallidas, fortaleciendo la confianza en garantías y devoluciones. Además, impulsa la transformación digital de la empresa y deja el camino listo para integrar, en el futuro, un canal de e-commerce o una plataforma de soporte para sus clientes.

Financiera

La implementación del sistema de inventario es una inversión con retorno comprobable: la TIR proyectada es del 20 %, superior a la tasa de descuento exigida del 12 %, por lo que el proyecto resulta atractivo. En la operación, reduce pérdidas por errores de inventario y mermas, disminuye ventas fallidas y protege el capital de trabajo. La automatización de reportes y alertas de stock optimiza el uso del personal al liberar horas administrativas. La visibilidad en tiempo real y los tableros de BI mejoran la toma de decisiones sobre compras, reposición y transferencias, elevando la rotación saludable y reduciendo el sobreinventario. Además, la arquitectura de microservicios con despliegue contenedorizado garantiza escalabilidad: permite agregar nuevos módulos sin invertir en más hardware y con costos predecibles. En conjunto, la solución evita quiebres, incrementa ingresos atendidos y mejora el margen al reducir desperdicios y tiempos ociosos.

Capítulo II

Marco teórico

Los sistemas de inventario son herramientas clave para administrar con precisión los bienes y productos de una organización. Controlan entradas y salidas, mantienen registros al día y aseguran la disponibilidad.

Inventario periódico

Se actualiza en intervalos definidos (semanal, mensual). Es simple y barato, pero no refleja el stock en tiempo real ni detecta quiebres a tiempo.

Inventario permanente o automatizado

Registra cada movimiento en el momento (ventas, compras, devoluciones, transferencias). Es ideal para múltiples sedes y alto volumen, mejora la trazabilidad y la toma de decisiones.

Figura 1: Resumen de VAN y TIR

Inversion Total	20000
Inversion Inicia	8000
2 Inversion	4000
3 Inversion	4000
4 Inversion	4000

Horizonte en meses

Flujo de Egresos	
Año	Egresos
1	S/ 12,000.00
2	S/ 11,000.00
3	S/ 10,000.00
4	S/ 12,000.00
5	S/ 14,000.00

Flujo de Ingresos	
Año	Ingresos
1	S/ 16,000.00
2	S/ 18,000.00
3	S/ 17,000.00
4	S/ 18,000.00
5	S/ 26,000.00

Flujo de Efectivo Neto	
Año	Efec.neto
1	S/ 4,000.00
2	S/ 7,000.00
3	S/ 7,000.00
4	S/ 6,000.00
5	S/ 12,000.00

Flujo de Efectivo Neto	
Año	Efec.neto
0	-S/ 20,000.00
1	S/ 4,000.00
2	S/ 7,000.00
3	S/ 7,000.00
4	S/ 6,000.00
5	S/ 12,000.00

Tasa Descuento (K)	12%
TIR > K	
20% >	12%

TIR	20%
VAN	S/ 4,756.48

Rentable
Ganancias

Nota. con un VAN de 4756.48 y un TIR del 20 %.

Antecedentes

Metodología de Desarrollo

Scrum es una metodología ágil e iterativa basada en la empiricidad (transparencia, inspección y adaptación). El trabajo se organiza en sprints de 1 a 4 semanas, al final de los cuales se entrega un incremento funcional y potencialmente desplegable. Los roles clave son Product Owner (prioriza valor), Scrum Master (facilita y remueve impedimentos) y un equipo multidisciplinario autoorganizado. Se apoya en eventos fijos —Sprint Planning, Daily, Review y Retrospective— y en artefactos como el Product Backlog y la Definición de Hecho; las historias de usuario guían el desarrollo y la revisión continua ajusta el rumbo.

■ Elementos clave:

- Historias de usuario: Describen funcionalidades desde la perspectiva del cliente.
- Criterios de aceptación: Definen cuándo una historia se considera completa.
- Sprint backlog: Lista de tareas a desarrollar en cada sprint.
- MVP (Producto Mínimo Viable): Versión inicial funcional que permite validar la solución con usuarios reales.

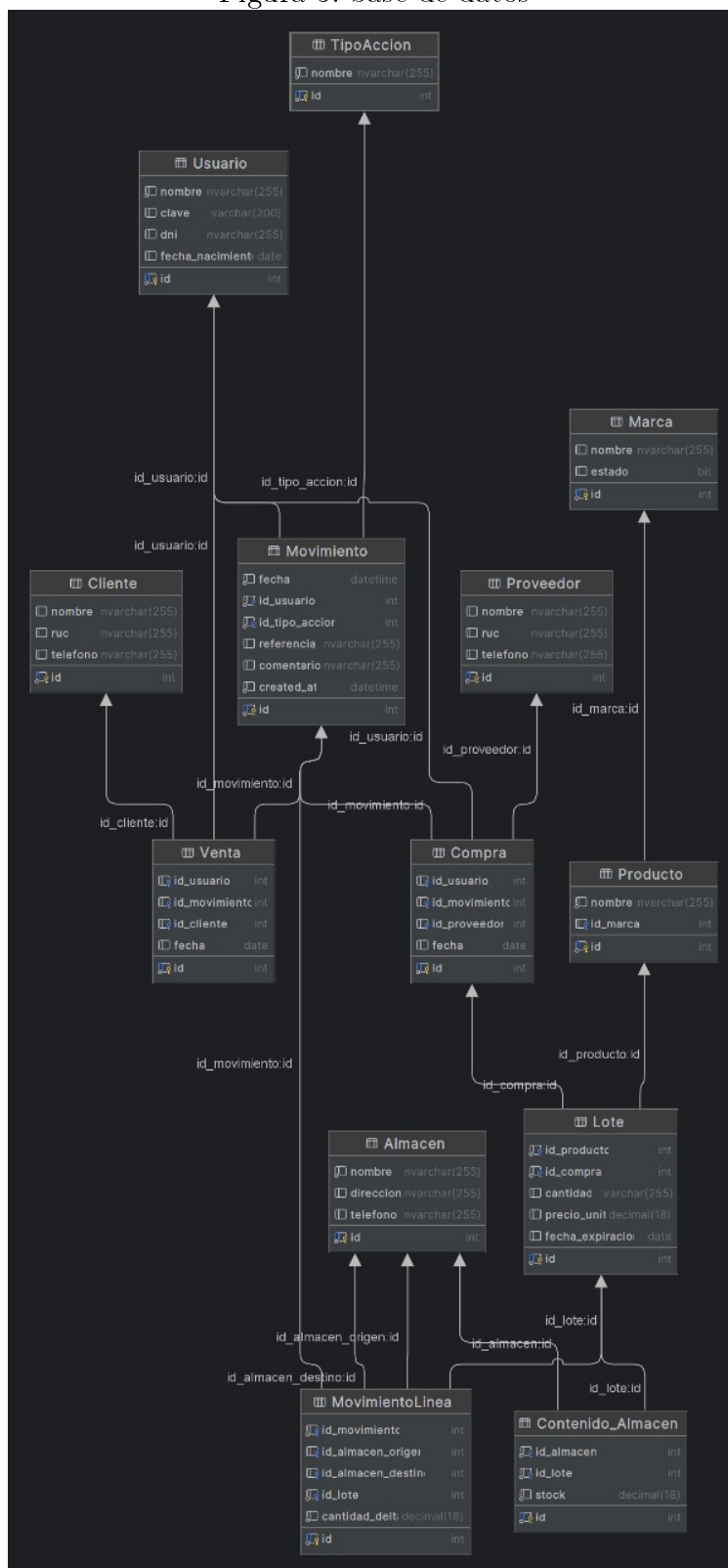
Figura 2: Carta gantt

Tareas	Estado	semana 1	semana 2	semana 3	semana 4	semana 5	semana 6	semana 7	semana 8	semana 9	semana 10	semana 11	semana 12	semana 13	semana 14	semana 15	semana 16
Iniciar Proyecto en Git	Culminado																
Planteamiento de la Problemática, objetivos y justificación	Culminado																
Diseñar Modelo Entidad-Relacion	Culminado																
Diseñar Maquetación Front-End	Culminado																
Script Inicial y Creación de BD	Culminado																
Desarrollo Back-End	Culminado																
Elaboración del Marco Teórico, Requerimientos Func. y No Func.	Culminado																
Elaboración Diagramas y Arquitectura	Culminado																
Desarrollo Front-End	En curso																
Programar Ruteo de la Página	Culminado																
Pruebas Unitarias de Endpoints	Culminado																
Integración y Pruebas Funcionales	En curso																
Corrección de Errores	En curso																
Elaboración Manuales de Usuario/Técnico, Conclusiones	En curso																
Refinamiento y Ajustes	En curso																
Preparación para Despliegue	En curso																
Cierre del Proyecto	En curso																

Nota. La carta Gantt muestra la planificación del proyecto durante 16 semanas

Diagrama de base

Figura 3: base de datos



Representación de la estructura de la base de datos

Capítulo III

Requerimientos

Funcionales

Los requerimientos funcionales especifican las acciones y comportamientos que un sistema debe realizar para cumplir con las necesidades del usuario y los objetivos del negocio. Definen las tareas, procesos y funciones que el software debe ejecutar, como la gestión de datos, interacciones con el usuario o respuestas ante eventos. Son esenciales para guiar el desarrollo y la validación del sistema, asegurando que cumpla con lo esperado y facilite la comunicación entre equipos y usuarios, evitando malentendidos durante el proyecto.

ID	Nombre	Descripción	Rol de usuario	Criterios de aceptación	Prioridad
RF-01	Crear usuario	Permite registrar un nuevo usuario ingresando nombre, DNI y fecha de nacimiento	Administrador	El usuario se almacena correctamente y es visible en la lista	Alta
RF-02	Editar usuario	Permite modificar los datos de un usuario existente	Administrador	Los cambios se reflejan al consultar el usuario	Alta
RF-03	Eliminar usuario	Permite borrar un usuario	Administrador	El usuario ya no aparece en la lista	Media
RF-04	Listar usuarios	Muestra los usuarios registrados con opción de búsqueda	Administrador	La búsqueda filtra correctamente	Alta
RF-05	Crear cliente	Permite registrar un nuevo cliente con nombre, RUC y teléfono	Administrativo	El cliente aparece en la lista general	Alta
RF-06	Editar cliente	Permite modificar los datos de un cliente existente	Administrativo	Cambios se muestran en la consulta	Alta
RF-07	Eliminar cliente	Permite dar de baja un cliente	Administrativo	El cliente ya no figura en búsquedas	Media

RF-08	Listar clientes	Visualiza los clientes registrados y permite buscarlos	Administrativo	Filtros y listados funcionales	Alta
RF-09	Crear proveedor	Permite registrar proveedores nuevos con nombre, RUC y teléfono	Administrativo	El proveedor se visualiza en la lista general	Media
RF-10	Editar proveedor	Permite editar información de proveedores existentes	Administrativo	Cambios son validados y almacenados	Media
RF-11	Eliminar proveedor	Permite eliminar un proveedor del sistema	Administrativo	El proveedor es eliminado de la lista	Baja
RF-12	Listar proveedores	Muestra y filtra la lista de proveedores registrados	Administrativo	Se pueden buscar proveedores existentes	Media
RF-13	Crear almacén	Permite registrar un nuevo almacén	Administrador	El almacén se almacena correctamente	Alta
RF-14	Gestionar productos y marcas	Permite creación, modificación y eliminación de productos y marcas	Administrativo 17	Operaciones se ejecutan satisfactoriamente	Alta

RF-15	Registrar lote	Permite registrar un lote para un producto y compra específica	Administrativo	El lote queda vinculado a su producto y compra	Alta
RF-16	Controlar inventario	Registra y actualiza stock de lotes en los almacenes, mostrando cantidades actuales	Administrador	El stock es exacto y se actualiza en base a movimientos registrados	Alta
RF-17	Registrar movimiento	Permite el registro de entrada, salida o traslado entre almacenes	Operador	El movimiento genera el ajuste esperado en los participantes	Alta
RF-18	Consultar movimientos	Listar todos los movimientos realizados, filtrando por fechas, usuario y tipo	Administrador	Filtros retornan resultados adecuados	Media
RF-19	Registrar compra	Permite almacenar información de una compra y asociarla a proveedor y lotes	Administrativo	Compra queda grabada y relacionada con lotes y proveedor	Alta
RF-20	Registrar venta	Permite almacenar información de una venta y asociarla a cliente y movimientos	Administrativo	Venta queda registrada y asocia los datos correspondientes	Alta

No Funcionales

Los requerimientos no funcionales definen cómo debe comportarse un sistema, abordando aspectos como seguridad, rendimiento y usabilidad. Son clave para garantizar su calidad, confiabilidad y una buena experiencia de usuario, por lo que no deben pasarse por alto.

ID	Categoría	Descripción	Criterio de aceptación	Prioridad
RNF-01	Seguridad	El sistema debe requerir autenticación para acceder a todas las funcionalidades.	Ningún usuario no autenticado puede acceder a la aplicación	Alta
RNF-02	Seguridad	Los datos personales deben almacenarse cifrados.	La base de datos almacena información sensible encriptada	Alta
RNF-03	Rendimiento	El tiempo de respuesta ante consultas de usuarios no debe superar 2 segundos en el 95 % de los casos.	Reportes de uso muestran cumplimiento del estándar	Alta
RNF-04	Disponibilidad	El sistema debe estar disponible el 99.5 % del tiempo en horario laboral.	Logs de uptime validan el cumplimiento	Media
RNF-05	Usabilidad	La interfaz debe ser intuitiva y accesible para usuarios nuevos, con mensajes claros de error y éxito.	Encuestas de satisfacción mayores a 80 % en pruebas	Media
RNF-06	Escalabilidad	El sistema debe soportar ampliación a más almacenes y usuarios sin degradar el rendimiento.	Pruebas de carga mantienen rendimiento aceptable	Media

RNF-07	Mantenibilidad	El sistema debe documentar su código y procesos clave para facilitar su actualización y corrección.	Manual técnico y comentarios obligatorios en el código	Media
RNF-08	Compatibilidad	Debe funcionar correctamente en los navegadores modernos (Chrome, Firefox, Edge).	Pruebas de compatibilidad con los principales navegadores	Baja
RNF-09	Backup	Debe ser posible restaurar la información mediante backups automáticos diarios.	Simulacros de recuperación de datos correctos	Media

Metodología de desarrollo desplegada

Para lograr una buena gestión y ejecución del proyecto, se adoptó la metodología de desarrollo **scrum**. El cual, se fundamenta en un enfoque ágil e iterativo que promueve la entrega incremental de valor, la colaboración constante entre los miembros del equipo y la adaptación continua a los cambios. Esta metodología divide el trabajo en ciclos cortos llamados *Sprints*, en los que se planifican, desarrollan y revisan funcionalidades del producto de manera continua, asegurando así una mejora constante y una mayor satisfacción del cliente.

Se definieron los roles dentro del equipo y se implementaron los eventos característicos de Scrum, tales como:

- **Sprint Backlog:** Es el trabajo a realizar durante un sprint, definido durante las reuniones de Planificación de Sprint. Incluye el objetivo del Sprint (*Sprint Goal*) y los

elementos del Product Backlog seleccionados para ser completados en ese sprint.

- **Sprint:** Planificación de 1 semana en el que se desarrollaron ideas y se convierten en valor.
- **Planificación de Sprint:** En esta reunión se definieron los objetivos del Sprint y el trabajo a realizar durante el sprint. Por lo que, la duración de esta reunión duro en total 336.
- **Scrum Diario:** La reunión durara alrededor de 15 minutos y sera realizada diariamente a la misma hora con el Equipo de Desarrollo para agilizar el proceso de desarrollo.
- **Revisión de Sprint:** Esta reunión tiene una duración máxima de 4 horas para sprints de un mes.

Diagrama de clases

Figura 4: Diagrama de clases

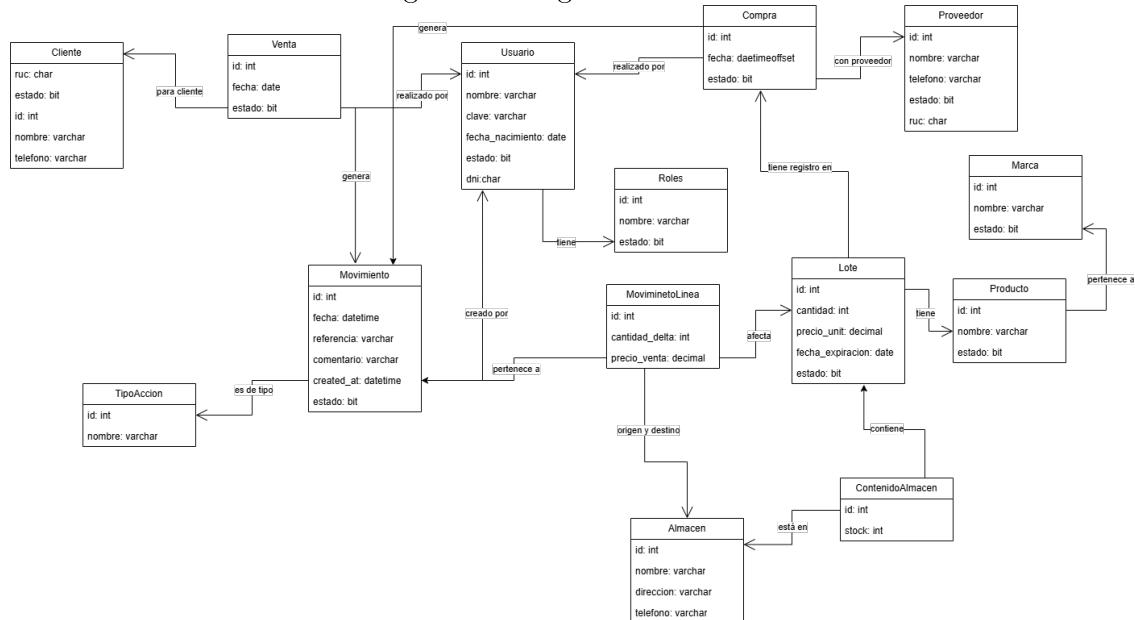


Diagrama UML de clases que representa la estructura y relaciones entre los objetos del sistema.

Entidades Maestras (Catálogos): Son las clases que almacenan datos centrales que rara vez cambian.

Usuario: Gestiona las cuentas que acceden al sistema, sus credenciales y su Rol.

Cliente: Almacena la información de los compradores (RUC, nombre).

Proveedor: Almacena los datos de los proveedores (RUC, nombre).

Producto: Es el artículo físico a inventariar (nombre), asociado a una **Marca**.

Almacén: Representa las ubicaciones físicas (dirección, teléfono) donde se guarda el stock.

Entidades Transaccionales (Procesos): Son las clases que registran las operaciones del negocio.

Compra y Venta: Son los procesos de negocio. Una **Venta** se asocia a un **Cliente** y una **Compra** a un **Proveedor**. Ambas son realizadas por un **Usuario** y, fundamentalmente, ambas generan un **Movimiento**.

Movimiento: Es el corazón del inventario. Actúa como un registro central de todas las operaciones (entradas, salidas, traslados). Es creado por un **Usuario** y se clasifica según un **TipoAccion**

Entidades de Inventario (Detalle): Son las clases que conectan los productos con las transacciones y el stock.

Lote: El sistema gestiona el inventario por lotes. Cada **Lote** tiene un `precio_unit` y `fecha_expiracion`, y está relacionado con un **Producto**.

MovimientoLinea: Es el detalle de un **Movimiento**. Especifica la cantidad de un **Lote** que se está moviendo.

ContenidoAlmacen: Es la clase que representa el stock real. Conecta un **Almacén** con un **Lote** y almacena la cantidad física (*stock*) disponible en esa ubicación.

Diagrama de base de datos

Figura 5: Diagrama de la base de datos

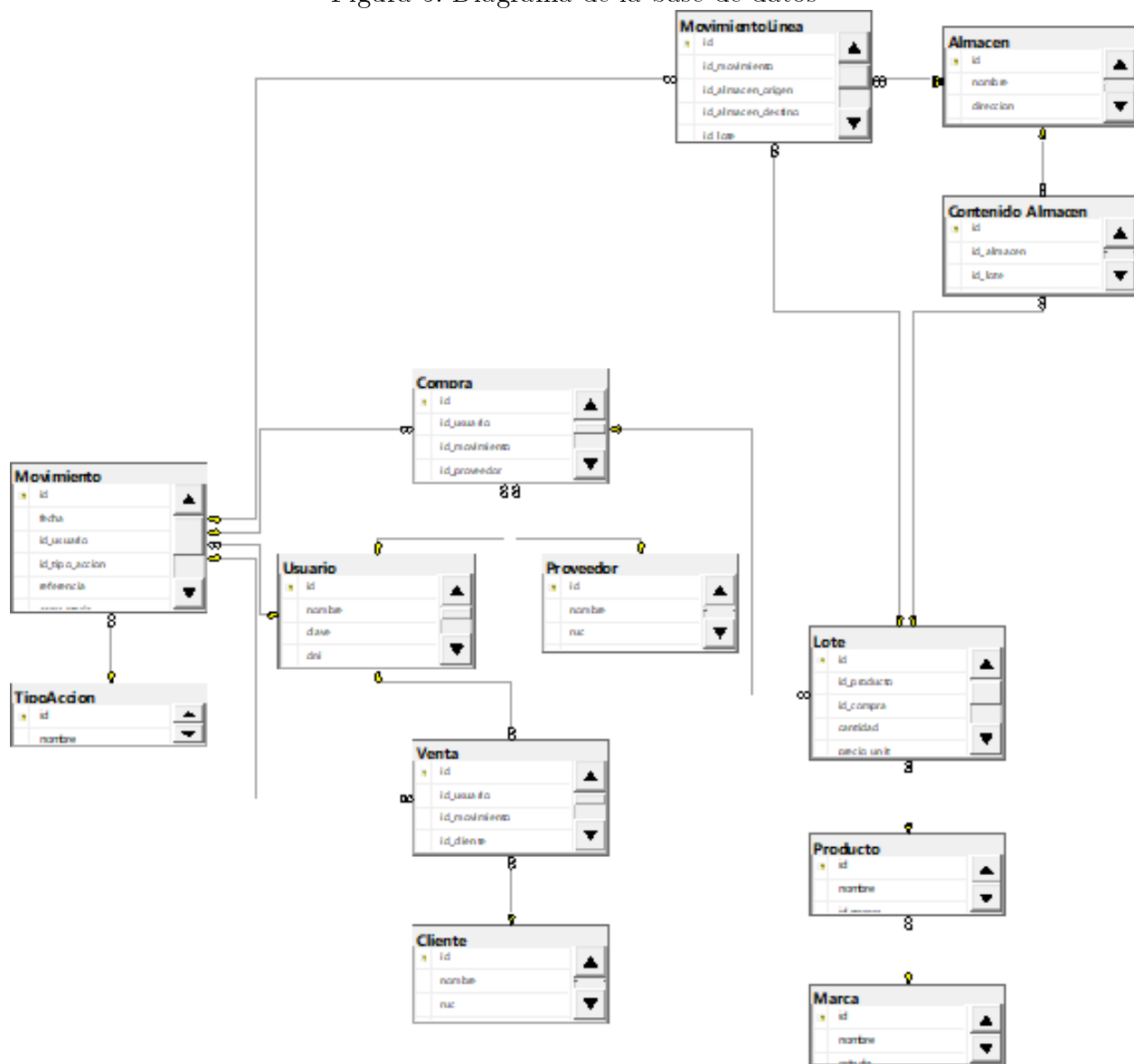


Diagrama que muestra la estructura de la base de datos y las relaciones entre tablas.

Diseño de entorno

Para el presente proyecto, se ha implementado una Arquitectura de 5 Capas, un modelo robusto que separa las responsabilidades del sistema en cinco niveles lógicos distintos: Modelo, Repositorio, Servicio, Controlador, y Configuración.

```
└─  org
  └─  example
    └─  prj_rest_control_almacen_hardware
      └─  Config
      └─  Controller
        ├── Almacen_Controller.java
        ├── Cliente_Controller.java
        ├── Compra_Controller.java
        ├── ContenidoAlmacen_Controller.java
        ├── Lote_Controller.java
        ├── Marca_Controller.java
        ├── Movimiento_Controller.java
        ├── MovimientoLinea_Controller.java
        ├── Producto_Controller.java
        ├── Proveedor_Controller.java
        ├── Roles_Controller.java
        ├── TipoAccion_Controller.java
        ├── Usuario_Controller.java
        └── Venta_Controller.java
      └─  Model
```

```
|— Almacen_Entity.java
|— Cliente_Entity.java
|— Compra_Entity.java
|— ContenidoAlmacen_Entity.java
|— Lote_Entity.java
|— Marca_Entity.java
|— Movimiento_Entity.java
|— MovimientoLinea_Entity.java
|— Producto_Entity.java
|— Proveedor_Entity.java
|— Roles_Entity.java
|— TipoAccion_Entity.java
|— Usuario_Entity.java
|— Venta_Entity.java
└─  Repository
    |— Almacen_Repository.java
    |— Cliente_Repository.java
    |— Compra_Repository.java
    |— ContenidoAlmacen_Repository.java
    |— Lote_Repository.java
    |— Marca_Repository.java
    |— Movimiento_Repository.java
    |— MovimientoLinea_Repository.java
    |— Producto_Repository.java
    |— Proveedor_Repository.java
    |— Roles_Repository.java
    |— TipoAccionRepository.java
    |— Usuario_Repository.java
```

```

    |— Venta_Repository.java
  └─ Service
    └─ impl
      |— Almacen_Service_Impl.java
      |— Cliente_Service_Impl.java
      |— Compra_Service_Impl.java
      |— ContenidoAlmacen_Service_Impl.java
      |— Lote_Service_Impl.java
      |— Marca_Service_Impl.java
      |— Movimiento_Service_Impl.java
      |— MovimientoLinea_Service_Impl.java
      |— Producto_Service_Impl.java
      |— Proveedor_Service_Impl.java
      |— Roles_Service_Impl.java
      |— TipoAccionServiceImpl.java

```

```

    |— Usuario_Service_Impl.java
    |— Venta_Service_Impl.java
  |— Almacen_Service.java
  |— Cliente_Service.java
  |— Compra_Service.java
  |— ContenidoAlmacen_Service.java
  |— Lote_Service.java
  |— Marca_Service.java
  |— Movimiento_Service.java
  |— MovimientoLinea_Service.java
  |— Producto_Service.java
  |— Proveedor_Service.java
  |— Roles_Service.java
  |— TipoAccionService.java
  |— Usuario_Service.java
  |— Venta_Service.java
└─ PrjRestControlAlmacenHardwareApplication.java

```

El proyecto utiliza una arquitectura en capas que sirve como base, la cual se ha adaptado

específicamente para el desarrollo de APIs REST siguiendo el patrón Model-View-Controller (MVC). En este enfoque, la función tradicional de la "Vista.^{en} MVC se reemplaza por la generación de respuestas en formato JSON, adecuándose así a la naturaleza de las APIs.

La implementación de esta arquitectura se basa en una clara separación de responsabilidades, donde cada capa cumple un rol específico y bien definido. Esto facilita la organización del proyecto, así como el mantenimiento y las pruebas del código.

Estructura y Patrones de Diseño Aplicados

La estructura del proyecto se divide en las siguientes capas, aplicando patrones de diseño específicos:

- **Capa de Modelo (Model):** Se encarga de representar las entidades de la base de datos como objetos Java. Para ello, utiliza el patrón Entity, mediante clases anotadas con JPA (`@Entity`), que mapean directamente las tablas en SQL Server y contienen únicamente la estructura de los datos.
- **Capa de Repositorio (Repository):** Proporciona el acceso directo a la base de datos para realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar). Esta capa aplica el patrón Repository, implementado mediante interfaces que extienden `JpaRepository`, lo que abstrae la lógica de persistencia y permite manipular los datos sin exponer los detalles de la base a otras capas.
- **Capa de Servicio (Service):** Aquí se encuentra la lógica de negocio: se realizan validaciones y se coordinan las operaciones necesarias llamando a los repositorios. Se aplica el patrón Service Layer, complementado con la inyección de dependencias (*Dependency Injection*) mediante `@Autowired` para integrar los repositorios requeridos.
- **Capa de Controlador (Controller):** Funciona como la interfaz de comunicación del sistema. Recibe las solicitudes HTTP externas, las dirige a la capa de servicio correspondiente y construye las respuestas HTTP. Emplea el patrón Controller, usando

anotaciones de Spring Boot (`@RestController`, `@GetMapping`, etc.) para exponer los endpoints REST.

- **Capa de Configuración (Config):** Gestiona configuraciones globales y transversales del sistema, tales como la configuración CORS, la seguridad con JWT y la documentación de la API (Swagger). Se basa en el patrón Configuration, utilizando anotaciones como `@Configuration` y `@Bean` para definir elementos y componentes gestionados por el contenedor de Spring.

Dependencia e Inyección

La interacción entre las capas se gestiona de manera rigurosa mediante la Inyección de Dependencias que proporciona Spring Boot. Las dependencias fluyen en una sola dirección: el controlador depende del servicio, y el servicio depende del repositorio.

Esta arquitectura aporta varios beneficios clave para el desarrollo:

- **Mantenibilidad:** Los cambios específicos, como modificaciones en la base de datos o en la lógica de negocio, afectan únicamente a la capa correspondiente sin propagarse innecesariamente al resto del sistema.
- **Testabilidad:** Cada capa puede ser probada de forma independiente mediante pruebas unitarias, facilitando la identificación y corrección de errores.
- **Escalabilidad y reutilización:** Es sencillo incorporar nuevas funcionalidades, y la lógica de negocio contenida en los servicios puede ser reutilizada por múltiples controladores.
- **Claridad:** La estructura de carpetas refleja intuitivamente la separación de responsabilidades, lo que mejora la comprensión y el mantenimiento del código.

Codificación

Capa de Modelo

Figura 6: MovimientoLinea

```

@Data
@Entity
@Table(name = "MovimientoLinea", schema = "dbo")
public class MovimientoLinea_Entity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false)
    private Integer id;

    @NotNull
    @ManyToOne(optional = false)
    @JoinColumn(name = "id_movimiento", nullable = false)
    private Movimiento_Entity idMovimiento;

    @ManyToOne
    @JoinColumn(name = "id_almacen_origen")
    private Almacen_Entity idAlmacenOrigen;

    @ManyToOne
    @JoinColumn(name = "id_almacen_destino")
    private Almacen_Entity idAlmacenDestino;

    @NotNull
    @ManyToOne(optional = false)
    @JoinColumn(name = "id_lote", nullable = false)
    private Lote_Entity idLote;

    @NotNull
    @Column(name = "cantidad_delta", nullable = false)
    private Integer cantidadDelta;

    @Column(name = "Precio_Venta", precision = 18)
    private BigDecimal precioVenta;

}

```

Modelo backend correspondiente a la entidad MovimientoLinea_Entity.

La Capa de Modelo representa el núcleo de la persistencia de datos. Su función principal es mapear la tabla física de la base de datos (`MovimientoLinea`) a un objeto Java (`MovimientoLinea_Entity`), aplicando el patrón Entity con anotaciones JPA. Esta capa define la estructura de la entidad, que incluye una clave primaria autoincrementada, y utiliza anotaciones de Lombok para minimizar el código repetitivo, como getters y setters.

Además, gestiona las relaciones *uno a muchos* y *muchos a uno* (`@ManyToOne`), conectando la línea de movimiento con entidades clave como `Movimiento`, `Lote` y los almacenes de origen y destino. Esto asegura la integridad referencial y permite una carga eficiente de los datos relacionados dentro de la aplicación.

Capa de Repositorio

Figura 7: `MovimientoLineaRepository`

```
package org.example.prj_rest_control_almacen_hardware.Repository;

import org.example.prj_rest_control_almacen_hardware.Model.MovimientoLinea_Entity;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface MovimientoLineaRepository extends JpaRepository<MovimientoLinea_Entity> {
    List<MovimientoLinea_Entity> findAll();
    Optional<MovimientoLinea_Entity> findById(Long id);
}
```

Repositorio backend correspondiente a la entidad `MovimientoLinea_Entity`.

La Capa de Repositorio actúa como el enlace exclusivo entre la Capa de Servicio y la base de datos, implementando el patrón Repository. Su principal función es abstraer la lógica de persistencia, aislando así a la Capa de Servicio de los detalles específicos de las consultas SQL.

Al extender la interfaz `JpaRepository`, esta capa hereda automáticamente los métodos CRUD básicos (como `save`, `findById`, `findAll` y `deleteById`) para la entidad `MovimientoLinea_Entity`, sin necesidad de escribir código SQL manual. Esto permite que el servicio obtenga datos

mediante interfaces simples (por ejemplo, `repository.findAll()`) sin preocuparse por la complejidad de la sintaxis SQL.

Capa de Servicio

Figura 8: MovimientoLineaService

```
import java.util.List;
import java.util.Optional;

public interface MovimientoLinea_Service {
    //CRUD
    List<MovimientoLinea_Entity> findAll();
    MovimientoLinea_Entity save(MovimientoLinea_Entity movimientoLinea);
    String deleteById(Long id);
    Optional<MovimientoLinea_Entity> update(Long id, MovimientoLinea_Entity movimientoLinea);
    //Métodos específicos
    Optional<MovimientoLinea_Entity> findById(Long id);
}
```

```
@Service
public class MovimientoLinea_Service_Impl implements MovimientoLinea_Service {
    @Autowired
    private MovimientoLinea_Repository movimientoLinea_Repository;

    @Override
    public List<MovimientoLinea_Entity> findAll() { return movimientoLinea_Repository.findAll(); }

    @Override
    public MovimientoLinea_Entity save(MovimientoLinea_Entity movimientoLinea) {
        return movimientoLinea_Repository.save(movimientoLinea);
    }

    @Override
    public String deleteById(Long id) {
        movimientoLinea_Repository.deleteById(id);
        return "MovimientoLinea eliminado con éxito";
    }

    @Override
    public Optional<MovimientoLinea_Entity> update(Long id, MovimientoLinea_Entity movimientoLinea) {
        Optional<MovimientoLinea_Entity> movimientoLineaExistente = movimientoLinea_Repository.findById(id);

        if (movimientoLineaExistente.isPresent()) {
            MovimientoLinea_Entity movimientoLineaActualizado = movimientoLineaExistente.get();
            movimientoLineaActualizado.setIdMovimiento(movimientoLinea.getIdMovimiento());
            movimientoLineaActualizado.setIdAlmacenOrigen(movimientoLinea.getIdAlmacenOrigen());
            movimientoLineaActualizado.setIdAlmacenDestino(movimientoLinea.getIdAlmacenDestino());
            movimientoLineaActualizado.setIdLote(movimientoLinea.getIdLote());
            movimientoLineaActualizado.setCantidadDelta(movimientoLinea.getCantidadDelta());
            movimientoLineaActualizado.setPrecioVenta(movimientoLinea.getPrecioVenta());

            return Optional.of(movimientoLinea_Repository.save(movimientoLineaActualizado));
        }
        return Optional.empty();
    }
}
```

Servicio backend correspondiente a la entidad MovimientoLinea_Entity.

La Capa de Servicio es el motor de la lógica de negocio y la orquestación, siguiendo el patrón Service Layer. Su principal responsabilidad es centralizar todas las reglas, validaciones y cálculos antes de que los datos sean guardados o retornados. Por ejemplo, en esta capa se debe validar que el *cantidadDelta* de un producto no provoque un stock negativo, o calcular automáticamente el precio de venta.

La implementación del servicio utiliza la inyección de dependencias (`@Autowired`) para acceder al Repositorio y realiza las operaciones básicas como listar, guardar, actualizar y eliminar, tal como se definen en su interfaz. Esto garantiza una separación clara entre la lógica de negocio y la capa de acceso a datos.

Capa de Controlador

Figura 9: MovimientoLineaController

```
package org.example.prj_rest_control_almacen_hardware.Repository;

import org.example.prj_rest_control_almacen_hardware.Model.MovimientoLinea_Enti;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface MovimientoLinea_Repository extends JpaRepository<MovimientoLinea_Enti> {
    List<MovimientoLinea_Enti> findAll();
    Optional<MovimientoLinea_Enti> findById(Long id);
}
```

Controller backend correspondiente a la entidad MovimientoLinea_Entity.

La Capa de Controlador es la puerta de entrada de la API, exponiendo los endpoints RESTful al exterior y aplicando el patrón Controller. Su función es manejar las solicitudes HTTP entrantes (como GET, POST, PUT, DELETE) en la ruta `/api/movimiento-lineas`. Utiliza la inyección de dependencias para acceder a la Capa de Servicio y delega en ella la ejecución de la lógica de negocio.

El controlador no contiene lógica de negocio; su tarea es recibir el JSON enviado por el cliente, invocar el método correspondiente en el servicio, y transformar el resultado de

las entidades Java en una respuesta HTTP (JSON) con el código de estado adecuado (por ejemplo, 200 OK, 204 No Content, 404 Not Found).

El flujo de datos sigue la dirección de la inyección de dependencias (Controlador → Servicio → Repositorio): una petición HTTP llega al controlador, que pasa la validación y la lógica al servicio, este interactúa con el repositorio para ejecutar la consulta SQL, y el resultado regresa como una respuesta JSON al cliente.

Los patrones clave aplicados en la arquitectura son: Entity Pattern (JPA), Repository Pattern para la abstracción de datos, Service Layer Pattern para la lógica de negocio, y Dependency Injection que integra todas las capas.

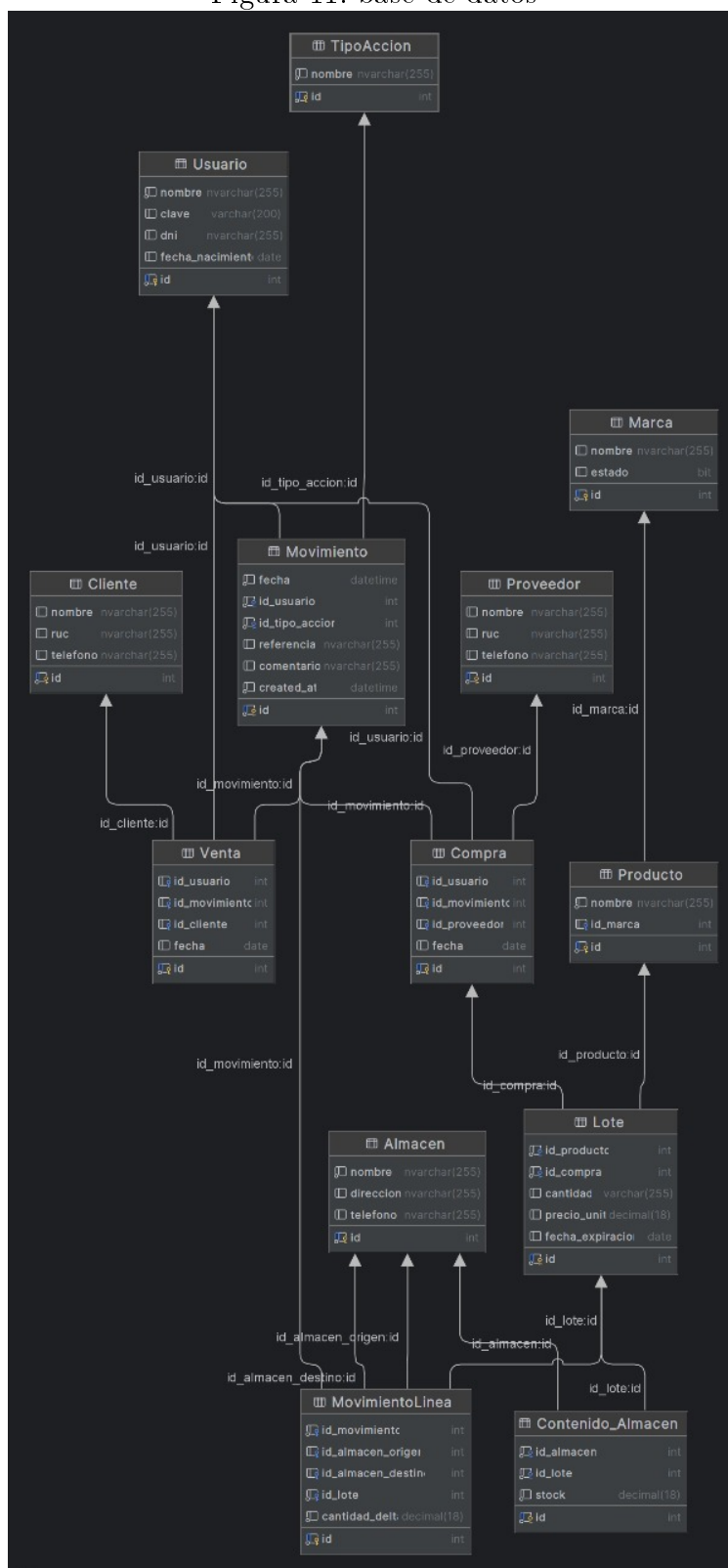
anexos

Figura 10: Carta gantt

Tareas	Estado	semana 1	semana 2	semana 3	semana 4	semana 5	semana 6	semana 7	semana 8	semana 9	semana 10	semana 11	semana 12	semana 13	semana 14	semana 15	semana 16
Iniciar Proyecto en Git	Culminado																
Planteamiento de la Problemática, objetivos y justificación	Culminado																
Diseñar Modelo Entidad-Relacion	Culminado																
Diseñar Maquetación Front-End	Culminado																
Script Inicial y Creación de BD	Culminado																
Desarrollo Back-End	Culminado																
Elaboración del Marco Técnico, Requerimientos Func. y No Func.	Culminado																
Elaboración Diagramas y Arquitectura	Culminado																
Desarrollo Front-End	En curso																
Programar Ruteo de la Página	Culminado																
Pruebas Unitarias de Endpoints	Culminado																
Integración y Pruebas Funcionales	En curso																
Corrección de Errores	En curso																
Elaboración Manuales de Usuario/Técnico, Conclusiones	En curso																
Refinamiento y Ajustes	En curso																
Preparación para Despliegue	En curso																
Cierre del Proyecto	En curso																

Nota. La carta Gantt muestra la planificación del proyecto durante 16 semanas

Figura 11: base de datos



Representación de la estructura de la base de datos

Figura 12: Diagrama de clases

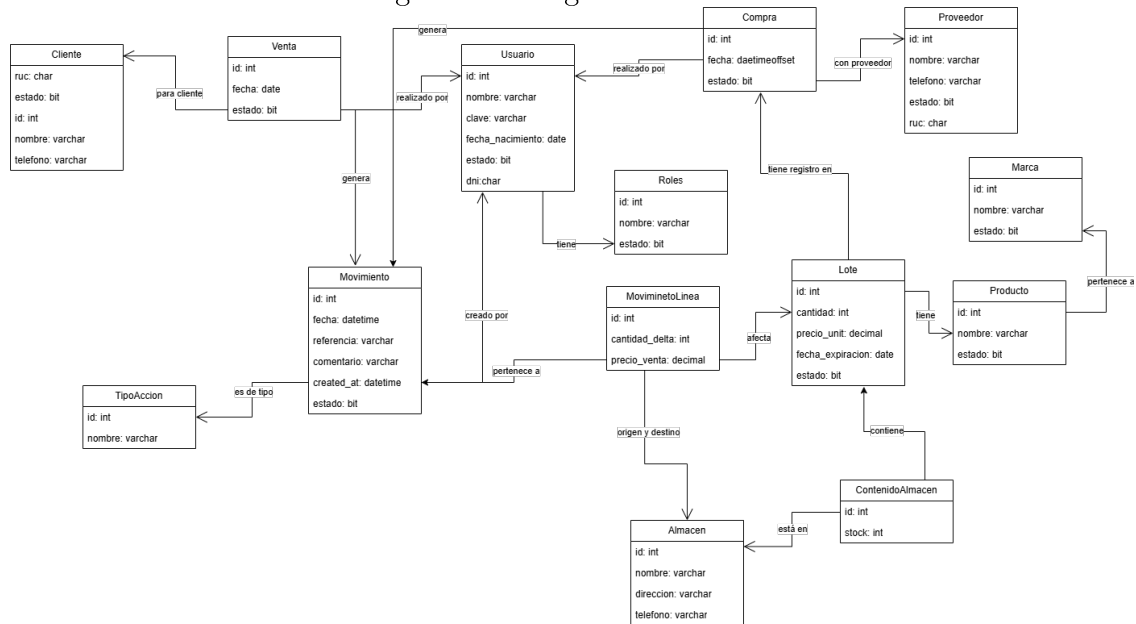


Diagrama UML de clases que representa la estructura y relaciones entre los objetos del sistema.

Figura 13: Diagrama de la base de datos

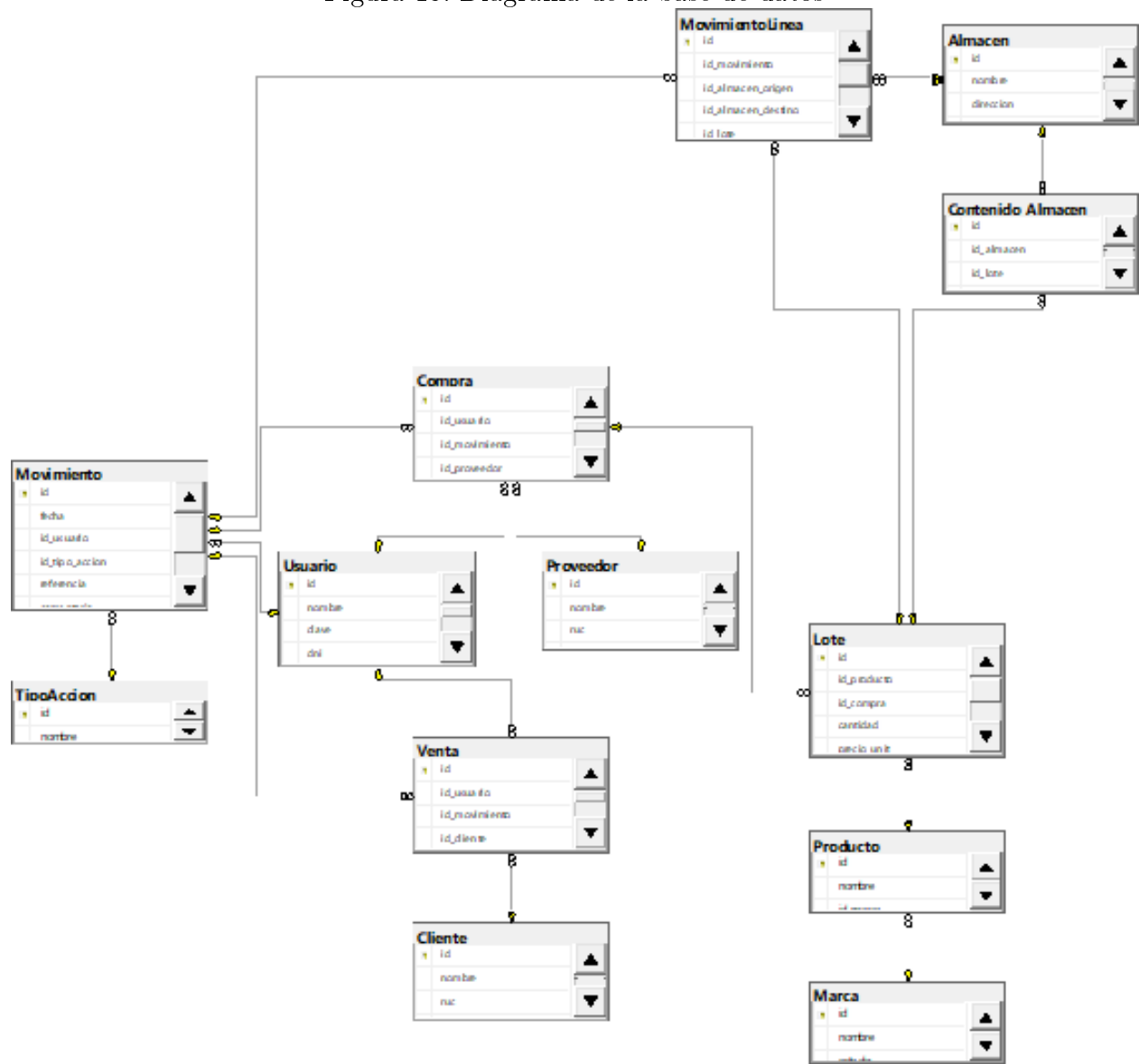


Diagrama que muestra la estructura de la base de datos y las relaciones entre tablas.